

The Cost of the ReAct Loop, Worked Out by Hand

Why an agent's bill grows with the *square* of its step count — every token counted, for one three-step customer-support trajectory.

What this document does. It takes the one tiny agent run from Module 0.0 — the *Customer-Support Resolver* that goes `crm_lookup` → `shipping_status` → `final_answer` in three steps — fixes the size (in tokens) of every message that flows through it, and then computes the prompt size, the token total, and the dollar cost at *every single step*. Nothing is hand-waved. Each number below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent. The trajectory in the module is quoted at `cost=$0.041`; we rebuild that figure from first principles and then show what it becomes at 5, 10, and 20 steps.

Where this sits. This is **Topic 1 of Module 0.0** — the headline number behind the autonomy-spectrum cost table (Chatbot 0.1¢ → Multi-Agent 22¢). Its companions: Module 0.1 (context-window pressure — the *same* growth, seen as a length limit instead of a bill) and Tracks 3.4 / 4.3 (cost engineering, where every lever exists to bend the curve derived here).

Concepts covered, in order: the ReAct loop as token accounting · base context vs. growing history · the per-step prompt $P(t) = B + (t-1)h$ · linear cost per step · the triangular number $\frac{N(N-1)}{2}$ · total cost as $O(N^2)$ · why an agent costs 10–100× a chatbot · what prompt caching changes (the constant) and what it does not (the shape).

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one run, small enough to count by hand

A ReAct agent is a loop. Each turn the model (i) reads everything that has happened so far, (ii) emits a *thought* and a tool *action*, and (iii) the runtime runs the tool and appends the *observation*. The loop repeats until the model calls `final_answer`. The pseudo-code from the module is just this:

```
1 def react_loop(goal, tools, max_steps=10):
2     history = []
3     for step in range(max_steps):
4         thought, action = llm.decide(goal, history, tools) # re-reads ALL history
5         if action.name == "final_answer":
6             return action.args["answer"]
7         observation = tools[action.name].run(**action.args)
8         history.append((thought, action, observation)) # history grows
9     raise RuntimeError("agent exceeded step budget")
```

The one fact that drives every number in this document is on line 4: `llm.decide` is *stateless*. The model keeps nothing between calls. To decide step t , the runtime must re-send the *entire* transcript so far. The transcript only grows. That is the whole story — the rest is arithmetic.

The quantities. We give every message a size in tokens. These are illustrative but realistic, and chosen round so you can follow the sums on paper. The mechanics are identical at any size — only the constants change.

Quantity	Symbol	Value here
Base context, re-sent every step (system prompt + 4 tool schemas + the goal)	B	2,000 tokens
Assistant message per step (thought + action)	a	300 tokens
Observation appended per step (tool result)	o	1,500 tokens
History added per completed step	$h = a + o$	1,800 tokens
Input price	p_{in}	\$3 / 1M tokens
Output price	p_{out}	\$15 / 1M tokens
Steps in this run	N	3

Notation. $P(t)$ is the number of *input* (prompt) tokens sent to the model on step t . Output tokens per step are the assistant message, a . Costs are tokens $\times$ price. We carry three decimals of a cent throughout.

Intuition

A ReAct agent has no memory except the transcript it re-sends. Think of it as a worker with total amnesia between questions: to ask them anything, you must first re-read them the entire case file. On step 1 the file is one page. By step 10 it is ten pages — and you pay to have it re-read *every single time*. The bill is not about how hard the model thinks; it is about how many times the growing file gets re-read.

2 Step 1 — what the model re-reads each turn

Before step t runs, the history holds the base context plus the $(t - 1)$ completed steps that came before it. Each completed step contributed $h = a + o$ tokens. So the prompt sent on step t is

$$P(t) = \underbrace{B}_{\text{fixed}} + \underbrace{(t - 1)h}_{\text{grows with the step}}. \quad (1)$$

This is a straight line in t : it starts at B (step 1 has no history yet) and climbs by h tokens every step. For our numbers, $P(t) = 2000 + (t - 1) \cdot 1800$.

Mini-example — this operation in isolation

The growing part is the only thing that matters at scale. On step 1, history = 0, so $P(1) = B = 2000$. On step 2 we have added one completed step, so $P(2) = 2000 + 1800 = 3800$. On step 3, two completed steps: $P(3) = 2000 + 3600 = 5600$. Notice the prompt on the *last* step is $5600/2000 = 2.8\times$ the prompt on the first. The later a step is, the more it costs — purely because it drags more history behind it.

3 Step 2 — the per-step bill

Now attach dollars. On step t the model reads $P(t)$ input tokens and writes a output tokens (on the final step the assistant message *is* the `final_answer`; we keep it at $a = 300$). The step cost is

$$c(t) = P(t) \cdot p_{\text{in}} + a \cdot p_{\text{out}}.$$

Working the three steps of the Customer-Support Resolver, with $p_{\text{in}} = \$3 \times 10^{-6}$ and $p_{\text{out}} = \$15 \times 10^{-6}$ per token:

Step	What happens	$P(t)$	in \$	out \$	$c(t)$	running \$
1	crm_lookup	2,000	0.0060	0.0045	0.0105	0.0105
2	shipping_status	3,800	0.0114	0.0045	0.0159	0.0264
3	final_answer	5,600	0.0168	0.0045	0.0213	0.0477
Total		11,400	0.0342	0.0135		\$0.0477

Sample check, step 2: input = $3800 \times 3 \times 10^{-6} = \0.0114 ; output = $300 \times 15 \times 10^{-6} = \0.0045 ; sum = $\$0.0159$. ✓ Three steps land at **\$0.0477** — the same order as the module’s quoted $\$0.041$ (the small gap is just our round token sizes; the *shape* is what we are after).

Two things to notice already. First, **input dominates**: $\$0.0342$ of input vs. $\$0.0135$ of output — because the prompt is re-read in full every step while the output stays small. Second, **the steps are not equal**: step 3 costs $2 \times$ step 1, even though the model “did” the same amount of work. The extra dollars bought nothing new — they paid to re-read history.

Intuition

The output is cheap and roughly constant; the *input* is what compounds. This inverts the naive mental model of “the model is expensive when it writes a lot.” In an agent loop, the model is expensive when it has *read* a lot — and it reads the whole transcript on every single turn.

4 Step 3 — the closed form

Three steps we can tabulate. To see the *shape*, sum Equation (1) over all N steps. The total input tokens are

$$\begin{aligned}
 T_{\text{in}}(N) &= \sum_{t=1}^N P(t) = \sum_{t=1}^N [B + (t-1)h] \\
 &= NB + h \sum_{t=1}^N (t-1) = NB + h \underbrace{\frac{N(N-1)}{2}}_{\text{triangular number}}.
 \end{aligned} \tag{2}$$

The base term NB is linear — it is the price of re-sending the fixed context N times. The second term carries the $h \cdot \frac{N(N-1)}{2}$ — it is *quadratic* in N , and it is the cost of history accumulating on itself. Output is the easy part: $T_{\text{out}}(N) = Na$, linear. Total dollars:

$$\boxed{\text{Cost}(N) = p_{\text{in}} \left[NB + h \frac{N(N-1)}{2} \right] + p_{\text{out}} Na} \tag{3}$$

Check against the table: $T_{\text{in}}(3) = 3 \cdot 2000 + 1800 \cdot \frac{3-2}{2} = 6000 + 1800 \cdot 3 = 6000 + 5400 = 11,400$.

✓ Exactly the column total above.

Mini-example — this operation in isolation

The sum $\sum_{t=1}^N (t-1) = 0 + 1 + 2 + \dots + (N-1)$ is a *triangular number*, $\frac{N(N-1)}{2}$. Stack a row of length 0, then 1, then 2, ... and you get a triangle — its area is half of an $N \times (N-1)$ rectangle. Tiny cases: $N = 3$ gives $0+1+2 = 3 = \frac{3 \cdot 2}{2}$; $N = 5$ gives $0+1+2+3+4 = 10 = \frac{5 \cdot 4}{2}$. Doubling N roughly *quadruples* this sum — that factor of four is the whole reason agent costs feel like they “run away.”

5 Step 4 — linear per step, quadratic in total

Two different growth rates live in this one loop, and conflating them is the classic mistake:

- The cost of *one* step, $c(t)$, grows **linearly** — step t costs about $p_{\text{in}} h t$ once t is large. Each step is a little more than the last.
- The cost of the *whole run*, $\text{Cost}(N)$, grows **quadratically** — because you pay that growing per-step price N times over. Linear price $\times$ linear count = quadratic bill.

Intuition

A step budget of “up to 20 steps” does not cost $20\times$ a one-step run. It costs closer to 20^2 -ish in the history term, because step 20 alone re-reads everything steps 1–19 produced. This is why “just let it loop a few more times” is the most expensive sentence in agent engineering — and why every cost lever later in the course (summarise the history, prune old observations, cap the step budget, early-exit) is really an attack on the same $\frac{N(N-1)}{2}$ term derived in Equation (2).

6 Step 5 — why an agent costs 10–100× a chatbot

The module’s autonomy table prices a chatbot at 0.1¢ and an agent at 8¢ — roughly $80\times$. Equation (3) explains the multiplier without any new ideas. A **chatbot** is the loop run with $N = 1$: one call, no accumulated history, cost $\approx p_{\text{in}} B + p_{\text{out}} a$. An **agent** runs the same call N times over a transcript that grows by h each turn. Holding our constants fixed and only varying N :

Mode	N	$T_{\text{in}}(N)$	final prompt $P(N)$	$\text{Cost}(N)$	vs. $N=1$
Chatbot-like	1	2,000	2,000	\$0.0105	1×
Short agent	3	11,400	5,600	\$0.0477	4.5×
Agent	5	28,000	9,200	\$0.1065	10×
Long agent	10	101,000	18,200	\$0.3480	33×
Runaway	20	382,000	36,200	\$1.2360	118×

From $N=1$ to $N=20$ the bill grows $\approx 118\times$ while the step count grew only $20\times$ — the extra factor of ~ 6 is the quadratic history term doing its work. The “10–100×” in the module’s table

is not a quirk of one model’s pricing; it is the triangular number, expressed in dollars. Sample check, $N = 5$: $T_{\text{in}} = 2000 \cdot 5 + 1800 \cdot \frac{5 \cdot 4}{2} = 10,000 + 18,000 = 28,000$, so input = \$0.084, output = $5 \cdot 300 \cdot 15 \times 10^{-6} = \0.0225 , total \$0.1065. ✓

Watch out

The same column also forecasts a *wall*, not just a bill. At $N = 20$ the final prompt $P(20)$ is $\sim 36,200$ tokens and still climbing by 1,800 a step. Long before the cost becomes unbearable, $P(t)$ collides with the model’s context window. So the loop has two independent failure modes from one cause: the **cost** ceiling (this document) and the **context** ceiling (Module 0.1). Both are the line $P(t) = B + (t - 1)h$ — read once as dollars, once as length.

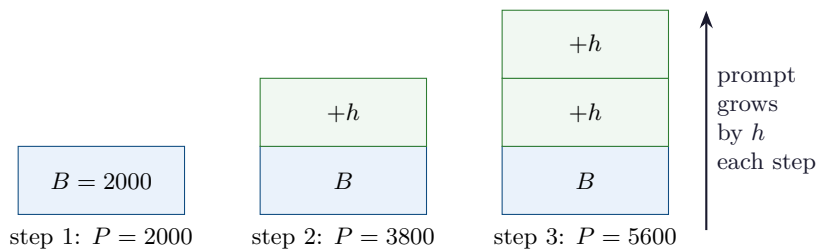
7 What bends this curve — and what does not

Prompt / KV caching changes the *constant*, not the *shape*. If the provider caches the unchanged prefix of the prompt, the re-sent base B and old history are billed at a discount (often $\sim 10\%$ of the input price) instead of full freight. That can cut the input term several-fold — but it still scales as $\frac{N(N-1)}{2}$, just with a smaller coefficient. Caching makes the agent cheaper; it does not make it linear.

History compression attacks the shape directly. Summarising old steps, or dropping stale observations, shrinks h (or caps it), so the quadratic term stops compounding. This is the lever the rest of the course leans on hardest.

What never changes: as long as the model is stateless and the transcript grows, total cost carries an $O(N^2)$ term. You can shrink its constant or cap its N ; you cannot remove its shape without changing one of those two facts.

8 The accounting, drawn



Each turn re-sends the blue base B plus one more green slab of history h . The blue is paid N times (linear); the green stack is a triangle (quadratic). The picture *is* Equation (2).

9 Cost cheat-sheet

Prompt sent on step t	$P(t) = B + (t - 1)h$
Cost of step t	$c(t) = P(t)p_{\text{in}} + ap_{\text{out}}$
Total input tokens	$T_{\text{in}}(N) = NB + h \frac{N(N-1)}{2}$
Total output tokens	$T_{\text{out}}(N) = Na$
Total cost	$\text{Cost}(N) = p_{\text{in}} \left[NB + h \frac{N(N-1)}{2} \right] + p_{\text{out}} Na$
Dominant growth	$O(N^2)$ in the history term $h \frac{N(N-1)}{2}$
Chatbot special case	$N = 1 \Rightarrow \text{Cost} = p_{\text{in}}B + p_{\text{out}}a$

10 An honest word about these numbers

The token sizes here ($B = 2000$, $h = 1800$) are illustrative round numbers, picked so the sums are followable by hand — exactly as the transformer-by-hand document used a 4-dimensional model that no one would ship. Your real B and h depend on your system prompt, how many tools you expose, and how chatty your tool outputs are; a verbose observation can make h several times larger than B , steepening the curve. What is *not* illustrative is the structure: a stateless model over a growing transcript pays $NB + h \frac{N(N-1)}{2}$ input tokens, full stop. Change the constants and the dollar figures move; the quadratic shape does not. That shape is the thing to carry into every later module — it is why agents have budgets, step caps, and history compression at all.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the per-step table, the closed-form totals, and the scaling table; change B , a , o , or the prices to explore your own agent's economics.

```

1  # react_cost.py -- reproduces every number in "The Cost of the ReAct Loop".
2  B = 2000 # base context re-sent every step (system + tools + goal), tokens
3  a = 300 # assistant message per step (thought + action), tokens -> also output
4  o = 1500 # observation appended per completed step, tokens
5  h = a + o # history added per completed step = 1800
6  P_IN = 3e-6 # input price, $ per token ($3 / 1M)
7  P_OUT = 15e-6 # output price, $ per token ($15 / 1M)
8
9  def prompt_tokens(t): # P(t) = B + (t-1)*h (t is 1-indexed)
10     return B + (t - 1) * h
11
12  def step_cost(t): # c(t) = P(t)*p_in + a*p_out
13     return prompt_tokens(t) * P_IN + a * P_OUT
14
15  def total_cost(N): # closed form, Eq. (3)
16     t_in = N * B + h * (N * (N - 1)) // 2
17     t_out = N * a
18     return t_in, t_out, t_in * P_IN + t_out * P_OUT
19
20  # --- per-step table for the 3-step trajectory ---
21  running = 0.0
22  for t in range(1, 4):
23     running += step_cost(t)
24     print(f"step {t}: P={prompt_tokens(t):5d} c={step_cost(t):.4f} running={running:.4f}")

```

```
25
26 # --- closed-form + scaling table ---
27 for N in (1, 3, 5, 10, 20):
28     t_in, t_out, cost = total_cost(N)
29     print(f"N={N:2d} T_in={t_in:7d} P(N)={prompt_tokens(N):6d} cost=${cost:.4f}")
30
31 # Expected (float64, exact):
32 # step 1: P= 2000 c=0.0105 running=0.0105
33 # step 2: P= 3800 c=0.0159 running=0.0264
34 # step 3: P= 5600 c=0.0213 running=0.0477
35 # N= 1 T_in= 2000 P(N)= 2000 cost=$0.0105
36 # N= 3 T_in= 11400 P(N)= 5600 cost=$0.0477
37 # N= 5 T_in= 28000 P(N)= 9200 cost=$0.1065
38 # N=10 T_in= 101000 P(N)= 18200 cost=$0.3480
39 # N=20 T_in= 382000 P(N)= 36200 cost=$1.2360
```

— end of document —